Design and Development of a Lightweight Question-Generation LLM from Scratch

End-to-End Implementation of a Decoder-Only Transformer

Kamal Nayan Kumar (23BDS026)

Altaf Raja (23BCS011)

Rahul Patel (23BDS047)

Vijaypal Singh Rathore (23BDS067)

Project Supervisor: Dr. Krishnendu Ghosh

https://github.com/Kamal-Nayan-Kumar/SLM

Project Overview

- **Goal:** Build a specialized, efficient language model for educational use without relying on expensive cloud APIs.
- **Architecture:** 114.1 million parameter decoder-only transformer.
- **Training:** Pretrained from scratch on 3B tokens; fine-tuned for specialized question generation.
- **Result:** A locally deployable SLM that generates semantically accurate reasoning questions.

At a Glance

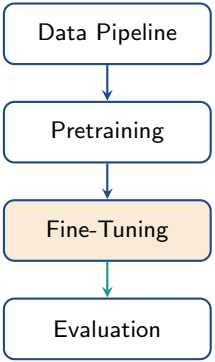
114M	**3 Billion**	**88,275**	**0.89+**
Parameters	Pretrain Tokens	SFT Samples	BERTScore F1

Why Build an SLM for Education?

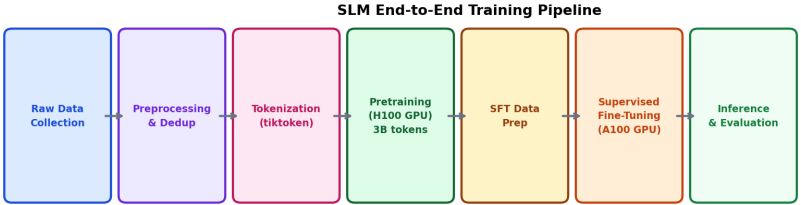
- **The Big Tech Bottleneck:** Current state-of-the-art models (e.g., GPT-4) require API keys, huge cloud dependency, and ongoing costs.
- **Privacy & Control:** Educational datasets may be sensitive; sending data to external APIs poses privacy and compliance risks.
- **Over-parameterization:** Generating targeted educational questions does not strictly require a 70B+ parameter model.
- **Solution:** An open, fully transparent small language model (SLM) optimized purely for offline, local generation.

Core Objectives

1. **Educational Completeness:** Implement the full neural lifecycle from data preparation to local inference.
2. **Practical Compactness:** Train efficiently within sensible hardware bounds (H100/A100) but deploy seamlessly on consumer hardware.
3. **Task Specialization:** Shift the model from predicting text to generating structured questions via rigorous fine-tuning.
4. **Modern Architecture:** Adopt research-proven efficiencies to punch above its weight class.



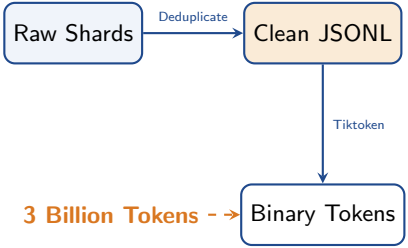
End-to-End Training Pipeline



Pretraining Data Engineering

Objective: Provide the model a robust foundational understanding of spelling, logic, facts, and structure.

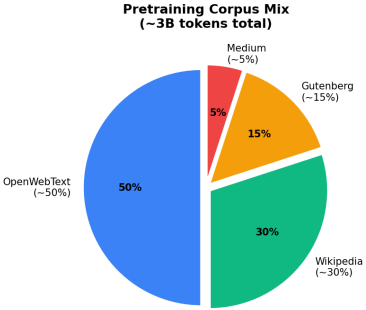
- **Cleaning & Filtering:** MinHash deduplication removing ∼15-20% noise, HTML stripping, Unicode normalization.
- **Tokenization:** Processed with OpenAI's `tiktoken` to condense cleaned text into memory-mapped arrays.



The 3 Billion Token Recipe

To prevent domain overfitting, the pretraining corpus was heavily blended across four distinct structures:

- **OpenWebText (50%):** General vocabulary and diverse internet discussion.
- **Wikipedia (30%):** Factual, encyclopedic knowledge.
- **Project Gutenberg (15%):** Long-form context and complex sentence structures.
- **Medium Articles (5%):** Conversational explanatory logic.



Model Architecture (114M Parameters)

- **Type:** Autoregressive decoder-only transformer.
- **Scale:** 12 Layers, 768 hidden dimension.
- **Context:** 4,096-token maximum sequence length window.
- **Vocabulary:** 50,257 tokens + 3 custom instruction tokens (50,260 total).
- **Weight Tying:** Shared embedding and prediction heads to dynamically stretch scale.

Architecture Strategy Comparison

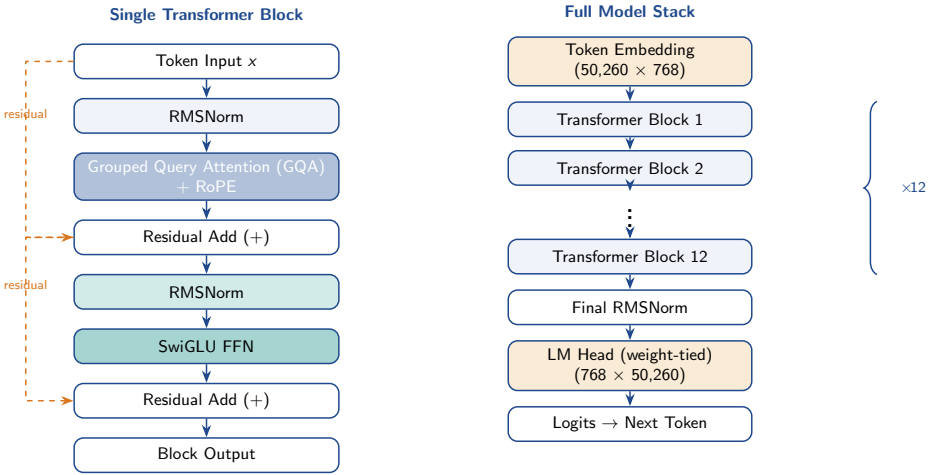
Decoder-only (Ours)

- No cross-attention overhead
- KV-cache for fast inference
- Native generation format
- Modern: GQA, RoPE, SwiGLU

Encoder-Decoder (T5, BART)

- Established QG baseline
- Separate encoder / decoder
- Larger parameter budgets
- Heavier inference overhead

Transformer Architecture Details

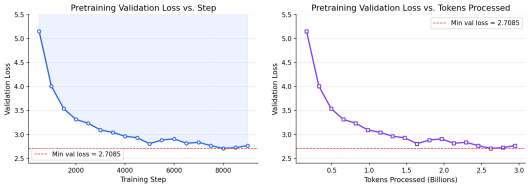


State-of-the-Art Efficiencies

- **Grouped Query Attention (GQA):** Pairs 12 Query heads with 4 Key/Value heads. Dramatically reduces KV-cache memory during inference.
- **Rotary Positional Embeddings (RoPE):** Encodes continuous relative token distances seamlessly compared to absolute embeddings limits.
- **SwiGLU FFN:** Replaces legacy ReLU. The gated linear unit vastly improves learning representation capacity per-parameter limit.
- **RMSNorm:** Pre-computation normalization strategy that is computationally cheaper and more mathematically stable than LayerNorm.

Pretraining Phase Computations

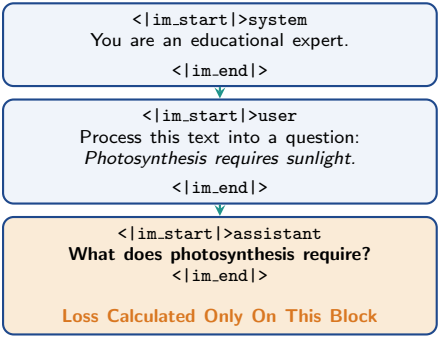
- **Hardware:** 1× NVIDIA H100 GPU (Lightning AI).
- **Compute Time:** Approximately 8 hours running 3 Billion tokens iteratively.
- **Hyperparameters:** `bfloat16` precision, gradient checkpointing enabled, cosine learning rate decay ($\eta_{\max}$ to $\eta_{\min}$).
- **Result:** Stable convergence from complete random noise to a valid representation (Final Loss $\sim$2.71).



Supervised Fine-Tuning (SFT) Rationale

From Continuer to Generator

- Pretrained models solely "predict". SFT shifts behavior to strictly adhere to instructions via the structured **ChatML Format**.
- **Loss Masking:** Critical technique. The Loss is entirely masked out for the prompt/context, meaning the model only updates weights to learn how to question, not text copying.

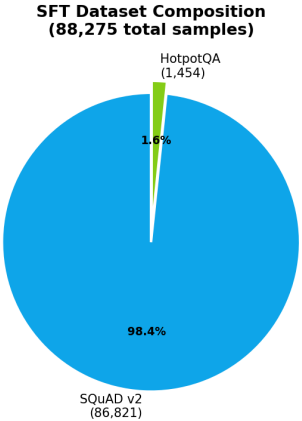


SFT Dataset Composition

Two Distinct Difficulty Levels

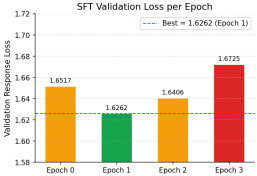
Compiled exactly **88,275** robust training samples focused intensely on precision educational generation formats.

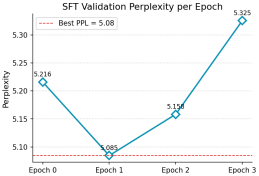
- **Factoid Short-Answer:** Extracted primarily via SQuAD v2 processing (98.4% of SFT blend).
- **Complex Multi-Hop:** Required synthesizing logical inferences across sentences via HotpotQA (1.6% of blend).



SFT Training Execution

- **Hardware:** 1× NVIDIA A100 GPU.
- **Compute Time:** Approximately 5 hours.
- **Strategy:** Tight learning rate (1e-5) acting on heavy weight decay to preserve the 3B-token pretraining while conforming to strict ChatML syntaxes.
- **Early Stopping:** Best semantic logic captured extremely early (Epoch 1) without syntax overfitting.





Fast Local Inference Engine

- **Local Capabilities:** Deployable seamlessly on local edge structures, natively supporting Apple Silicon (Mac M-series) processing.
- **Cache Intelligence:** KV-Caching Mechanism explicitly saves previously computed key-value tensors when walking long educational PDFs.
  - Drops computation per-head drastically without latency spikes.
- **Factual Sampling Parameters:**
  - Nucleus top-p bounds at 0.90 to reject absurd branches.
  - Deterministic temperature bounds natively set to $T = 0.3$.

Quantitative Evaluation Principles

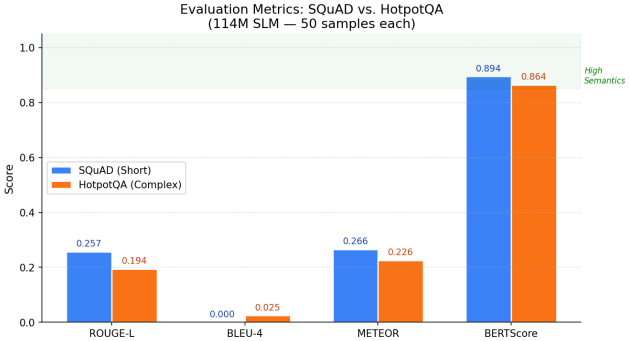
How Do We Measure A "Good" Question?

Evaluated strictly against completely held-out test data from SQuAD and HotpotQA sets.

- **Lexical (BLEU, ROUGE):** Checks if identical words copy over. Models often score poorly here naturally because there are many valid ways to phrase a test query.
- **Semantic (BERTScore):** Uses deep vector-embeddings to check if the generated question structure actually means the same thing as the ground-truth teacher-question.

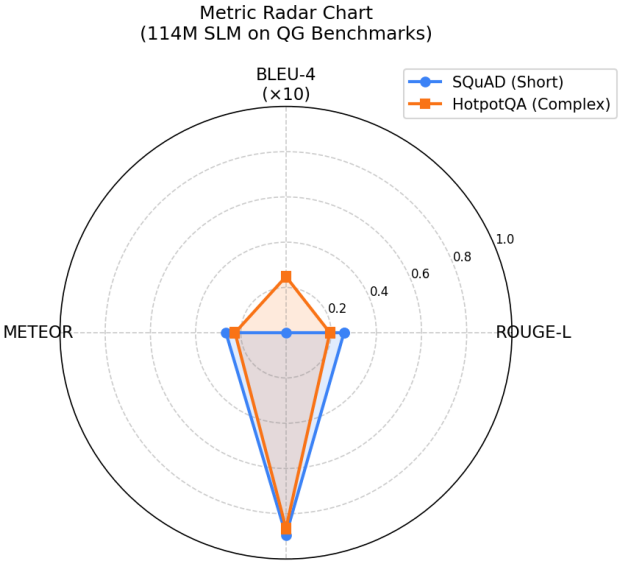
Quantitative Results

- Achieved an exceptional **BERTScore F1 of 0.89+** on Short Questions and **0.86+** on Complex reasoning.
- **BLEU-4** remains deliberately low, verifying that the SLM is generating autonomous queries, not merely string-matching via copy-pasting.



Multi-Metric Profile (Radar Pattern)

The Radar pattern proves clearly that the semantic intelligence of a deliberately fine-tuned 114M model is incredibly strong, despite the lack of direct lexical token overlapping (visual mismatch between BERTScore and BLEU).



Demonstration: Fact Extraction

Short-Answer query generation explicitly built around explicit context tracing.

```
python src/inference/generate.py  –  SLM Question Generator

$ python src/inference/generate.py --difficulty short
Model loaded from checkpoints/sft/best_model.pt  |  device: cpu

Demo 1 – Short Question (Biology)
DIFFICULTY: short  |  python src/inference/generate.py --difficulty short
Input Passage:
Photosynthesis is the process by which green plants, algae, and some bacteria convert
light energy—usually from the Sun—into chemical energy stored in glucose. Chlorophyll,
the green pigment found in chloroplasts, absorbs sunlight and uses its energy to combine
carbon dioxide from the air with water from the soil, producing glucose and oxygen as
byproducts. The overall chemical equation is: 6CO₂ + 6H₂O + light → C₆H₁₂O₆ + 6O₂.
Generated Question:
► Provide an example of a plant that uses the energy produced by photosynthesis to form water.
```

Demonstration: Complex Reasoning

Forced complex reasoning via synthesizing disparate paragraph inputs into a single combined query element.

```
python src/inference/generate.py  —  SLM Question Generator

$ python src/inference/generate.py --difficulty complex
Model loaded from checkpoints/sft/best_model.pt  |  device: cpu

Demo 3 — Complex Question (History)
DIFFICULTY: complex  |  python src/inference/generate.py --difficulty complex
Input Passage:
World War II began on September 1, 1939, when Nazi Germany, under Adolf Hitler,
invaded Poland. Britain and France declared war on Germany two days later. The war
expanded rapidly across Europe, North Africa, and eventually the Pacific after Japan
attacked Pearl Harbor in December 1941, drawing the United States into the conflict.
The war ended in Europe on May 8, 1945 (V-E Day), and in the Pacific on September 2,
1945 (V-J Day), after the United States dropped atomic bombs on Hiroshima and Nagasaki.
Generated Question:
► Provide an example of a significant event that occurred on September 2, 1945.
```

Conclusion & Future Scope

Project Achievements

- Fully constructed a modern, compact SLM from scratch without reliance on external APIs.
- Delivered 0.89 BERTScore semantics on restricted local compute arrays.
- Open-sourced the entire repository cycle.

Future Trajectories

- Extracting weights directly into ONNX/CoreML for friction-less Android/iOS integrations.
- Re-integrating DPO/RLHF algorithms to heavily refine multi-hop logic reasoning pathways.

Thank You!

Full implementation repository:
https://github.com/Kamal-Nayan-Kumar/SLM